



SWEN 221

A surreal, dreamlike landscape. In the foreground, a person stands on a dark, rocky outcrop, looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, and blue. A bright, glowing horizon line suggests a sunrise or sunset. In the upper right, a comet streaks across the sky, leaving a trail of light. The overall atmosphere is ethereal and otherworldly.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



Assertions -- contracts



Assert Keyword

- Added in Java 1.4
 - Basic use is extremely simple:
 - **assert** <condition>;
- Or
- **assert** <condition> : <message>;
 - Disabled by default. Use `java -ea MyProgram`
 - **assert** `x!=null`;
 - **assert** `x>=0`;
 - **assert** `x!=null`: "x can not be not null";
 - **assert** `x>=0`: "x can not be negative";

but also

- **assert** `isConsistent()`;
- **assert** `x!=null` : `ErrorHelpers.notNull("x","Reason")`;



```
void foo (A a) {  
    assert a!=null;  
    . . .  
}
```

Semantic

- `assert x!=null;`
 - if x is `null` and assertions are enabled, then
- `throw new AssertionError();//Unchecked Exception`
- `assert x!=null : "msg";`
 - if x is `null` and assertions are enabled, then
- `throw new AssertionError("msg");//Unchecked Exception`



```
void foo (A a) {  
    assert a!=null;  
    ...  
}
```


How to enable assertions

- Command line:
 - `java -ea MyMainClass`
 - `java -ea -jar MyExecutablejarFile`
- Eclipse (stupid repetitive mode):
 - Run → Run configurations → select your configuration → arguments → vm arguments → write “-ea”
- Eclipse (smart but invasive mode):
 - Window→Preferences→Java→Installed JREs
Select the JRE, press Edit button, inside “Default JRE Arguments” write “-ea”.
- To enable assertions only in a package (and sub-packages)
 - `-ea:namePackage...`
 - No space before the three dots!!!
- Even if Eclipse is in debug mode, even if you are using Junit or similar tools, assertions may not be automatically enabled.



Quiz - 1

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 1

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError

B "b is true"

C "b is false"

D "b is null"

E NullPointerException

F something else

Quiz - 1

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError



"b is true"

C

"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 1

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError



"b is true"

C

"b is false"



"b is null"



NullPointerException



something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```



AssertionError

B "b is true"

C "b is false"

D "b is null"

E NullPointerException

F something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```



AssertionError



"b is true"

C

"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```



AssertionError



"b is true"



"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```



AssertionError



"b is true"



"b is false"



"b is null"

E

NullPointerException

F

something else

Quiz - 2

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac -ea A.java  
>> java A
```



AssertionError



"b is true"



"b is false"



"b is null"



NullPointerException

F

something else
invalid flag: -ea

Quiz - 3

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 3

```
class A{  
    public static boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```

A AssertionError



"b is true"



"b is false"



"b is null"



NullPointerException



something else

Quiz - 4

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 4

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError

B "b is true"

C "b is false"

D "b is null"

E NullPointerException

F something else

Quiz - 4

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError



"b is true"

C

"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 4

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError



"b is true"



"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 4

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError



"b is true"



"b is false"

D

"b is null"



NullPointerException



something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError

B "b is true"

C "b is false"

D "b is null"

E NullPointerException

F something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError



"b is true"

C

"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError



"b is true"



"b is false"

D

"b is null"

E

NullPointerException

F

something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError



"b is true"



"b is false"



"b is null"

E

NullPointerException

F

something else

Quiz - 5

```
class A{  
    public static Boolean b;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError



"b is true"



"b is false"



"b is null"

E

NullPointerException



something else

Quiz - 6

```
class A{  
    public static Boolean b=true;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```

- A AssertionError
- B "b is true"
- C "b is false"
- D "b is null"
- E NullPointerException
- F something else

Quiz - 6

```
class A{  
    public static Boolean b=true;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError

B "b is true"

C "b is false"

D "b is null"

E NullPointerException

F something else

Quiz - 6

```
class A{  
    public static Boolean b=true;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java -ea A
```



AssertionError

B

"b is true"



"b is false"



"b is null"



NullPointerException



something else

Quiz - 7

```
class A{  
    public static Boolean b=true;  
    public static void main(String[] args) {  
        assert b;  
        System.out.println("b is "+b) ;  
    }  
}
```

```
>> javac A.java  
>> java A
```



AssertionError

B

"b is true"



"b is false"



"b is null"



NullPointerException



something else

Confusion:

Assertions



Testing

Kinds of testing

- Manual testing: manually run the program and look to the behavior with your miserable human eyes.
- Automatic testing: run a single case input→output and write a functionality that checks if the output is the expected output
Using (for example) `assertEquals(expected, ex1);`
- (Automatic) Property based testing on a fixed dataset: run the same checks on a large amount of inputs. (not discussed here)
- Property based testing on automatically generated data sets == Fuzz testing (not discussed here)
- **Runtime verification: using assertions inside of methods to verify method contracts (using `assert e;`)**
 - Runtime verification is kick-started from unit tests

Confusion:

Runtime verification



(Method) contracts



Assertions

Confusion:

Runtime verification

Normally, testing code is outside of the application. **Runtime verification** enhances normal tests by running extra checks inside of the running code



(Method) contracts



Assertions

Confusion:

Runtime verification

Normally, testing code is outside of the application. **Runtime verification** enhances normal tests by running extra checks inside of the running code

(Method) contracts

Those checks run before and after method bodies. They enforce methods follow a logical specification. Logical specification == contract

Assertions

Confusion:

Runtime verification



(Method) contracts



Assertions

Normally, testing code is outside of the application. **Runtime verification** enhances normal tests by running extra checks inside of the running code

Those checks run before and after method bodies. They enforce methods follow a logical specification. Logical specification == contract

Assertions are used as a concrete tool to run those checks. Since assertions can be enabled/disabled, we can disable them in production for performance reasons if needed.

Method contracts

**Every method has,
implicitly or explicitly,
a contract.**

```
int factorial(int n) {  
    if (n < 1) { return 1; }  
    return n * factorial(n - 1);  
}
```

**If method is called over acceptable parameters,
then a certain result is produced.**

Method contracts

**Every method has,
implicitly or explicitly,
a contract.**

```
int factorial(int n) {  
    assert n >= 0;  
    if (n < 1) { return 1; }  
    return n * factorial(n - 1);  
}
```

**If method is called over acceptable parameters,
then a certain result is produced.**

Modify the contract of the method; make it simpler!

Pre/Post conditions

- Checks at the start of the method execution are called preconditions
- It is a usage limitation: ensures your code is called as expected

Pre/Post conditions

- Checks at the start of the method execution are called preconditions
- It is a usage limitation: ensures your code is called as expected
- Checks at the end of method execution are called postconditions.
- It is a behavioral constraint: ensures your code behave as expected

Pre/Post conditions

- Checks at the start of the method execution are called preconditions
- It is a usage limitation: ensures your code is called as expected
- Checks at the end of method execution are called postconditions.
- It is a behavioral constraint: ensures your code behave as expected

Doing useful checks in the middle of the method execution is a good idea, just there is no a special name

```
void methodName (...) {  
    assert precondition();  
    try {  
        ...DoSomething...  
    }  
    finally { assert postcondition(); }  
}
```

How much code in assertions?

- Verification code is very important.
- As with regular code, you can organize it into with methods, classes, libraries..
- A balanced program with assertions can contain up to 50% of verification code
- In addition to testing.
- So, if the minimum code to encode the logic would be 1k lines, we would expect an additional 1k lines of tests and 1k lines of assertions-related code.

If the minimum code to encode the logic would be 1k lines, we would expect an additional 1k lines of tests and 1k lines of assertions-related code.

```
//MyCode.Java
class MyClass{
    int myMethod(..){
        assert x != null;
        assert allGood();
        int result;
        try{ return result=..; }
        finally{assert resGood(result);}
    }
    boolean allGood(){..}
    boolean resGood(int x){..}
}
```

Number of logic lines in green

~=

Number of Assertion lines in yellow

```
//MyCodeTest.Java
class TestMyClass{
    @Test void myClassIsBlue(..){
        MyClass c= new MyClass();
        int res= c.myMethod(..);
        assertEquals(25,res);
    }
}
```

Number of logic lines in green

~=

Number of Test lines in orange

Assertions -- Libraries

- Great for debug (much better than `System.err.println(..)`)
- Cooperate with private state! (preserve valid program state)
- Cooperate with documentation (active documentation)
- Make library usage safer!
- When a library is used in unacceptable way
 - `AssertionError` with an informative error message

Programmatically enabling assertions

- We can enable assertions programmatically. For example in the main. What do you think happens in the code below?

```
public class Main {  
    public static void main(String[]a){  
        ClassLoader.getSystemClassLoader()  
            .setDefaultAssertionStatus(true);  
        assert false:"Will it break here?";  
        System.out.println("Hello");  
        Other.method();  
    }  
}  
  
class Other{  
    static void method(){  
        assert false:"Or here?";  
        System.out.println("World");  
    }  
}
```



Programmatically enabling assertions

- We can enable assertions programmatically. For example in the main. What do you think happens in the code below?

```
public class Main {//Assuming a start with disabled assertions
    public static void main(String[]a){ //First the class Main is loaded
        ClassLoader.getSystemClassLoader()
            .setDefaultAssertionStatus(true);//for newly loaded classes only
        assert false:"Will it break here?"; //No error here
        System.out.println("Hello"); //Prints no problem
        Other.method(); //Loading class 'Other'
    }
}
class Other{
    static void method(){
        assert false:"Or here?"; //Error HERE :-)
        System.out.println("World");
    }
}
```

Testing for enabled assertions

- Once upon a time, JUnit was NOT enabling assertions automatically.
- At that time, having this 'test' was common.

```
@Test void assertsAreRunning() {  
    try{ assert false:"AAAH"; }  
    catch(AssertionError ae){ return; }  
    fail("Assertions are disabled");  
}
```

- Now you can still find it in some legacy code.
The concept can still be useful in case you have to work on older systems.

Contracts

A conceptual image featuring two devil figurines on a desk. One devil figurine is in the foreground, crouching and looking towards the right. The other is in the background, sitting on a small black pedestal and looking towards the left. They are positioned over a large sheet of paper with handwritten text, which appears to be a contract. A red and gold pen lies on the paper to the right of the background devil. The scene is set on a wooden desk with a warm, reddish-brown tone. The lighting is dramatic, with strong highlights and deep shadows.

Invariants

Contracts:

- Precondition: blame the user of your code
 - Debugging/testing support
 - Active documentation. The error can point to the documentation about your functionality
 - Can be an assert
 - Can be an `if(cond){ throw new IllegalArgumentException(..); }`
- Postcondition:
 - Debugging/testing support
 - Can be an assert
 - Can be another kind of 'Error'

Contracts:

- Invariant: some property that holds for all the instances of a type

```
record Positive(int x){ Positive{ assert x >= 0; } }
```

```
record Even(int x){ Even{ assert x % 2 == 0; } }
```

```
record Person(String name, Positive age){  
    Person{  
        assert name != null;  
        assert !name.isEmpty();  
        assert !name.contains("\n");  
    }  
}
```

Invariants on deeply mutable objects

//What is wrong here?

```
record Car(List<Wheel> wheels){ Car{ assert wheels.size() == 4; } }
```

...

```
new Car(List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel()));//OK
```

...

```
List<Wheel> wheels= List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel());  
Car bessie= new Car(wheels);//OK TOO
```

...

```
List<Wheel> wheels= new ArrayList<>(  
    List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel())  
);
```

```
Car rascal= new Car(wheels);//Is this ok?
```

Invariants on deeply mutable objects

//What is wrong here?

```
record Car(List<Wheel> wheels){ Car{ assert wheels.size() == 4; } }
```

...

```
new Car(List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel()));//OK
```

...

```
List<Wheel> wheels= List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel());  
Car bessie= new Car(wheels);//OK TOO
```

...

```
List<Wheel> wheels= new ArrayList<>(  
    List.of(new Wheel(), new Wheel(), new Wheel(), new Wheel())  
);
```

```
Car rascal= new Car(wheels);//Is this ok?
```

```
wheels.remove(2);//Nope!
```

```
assert rascal.wheels().size() == 4;//We can observe a broken invariant
```

Invariants on deeply mutable objects

```
record Car(List<Wheel> wheels){  
  Car{  
    wheels = List.copyOf(wheels); //No-op if it was already an unmodifiable copy  
    assert wheels.size() == 4;  
  }  
}
```

```
List<Wheel> wheels= ...;  
Car rascal= new Car(wheels); //here 'defensive cloning' happens  
  
//Be careful with Lists of Lists or other deeply mutable types
```

Invariants on deeply mutable objects

```
Wheel w= new Wheel();  
List<Wheel> wheels= List.of(w,w,w,w);  
Car rascal= new Car(wheels);
```

```
//Is this ok? The 4 wheels are actually the same single wheel 4 times.  
//Is this ok? Well... it really depends on the detail of the Wheel type..
```

Concluding

- Assertions: a fundamental tool to write better code!
- Precondition: amazing to make you code usable.
- Postconditions: great to find bugs.
- Invariants: great tool to understand code behavior. But can give a false sense of security if badly implemented.

Next time: Inner classes

